

Practical: 1

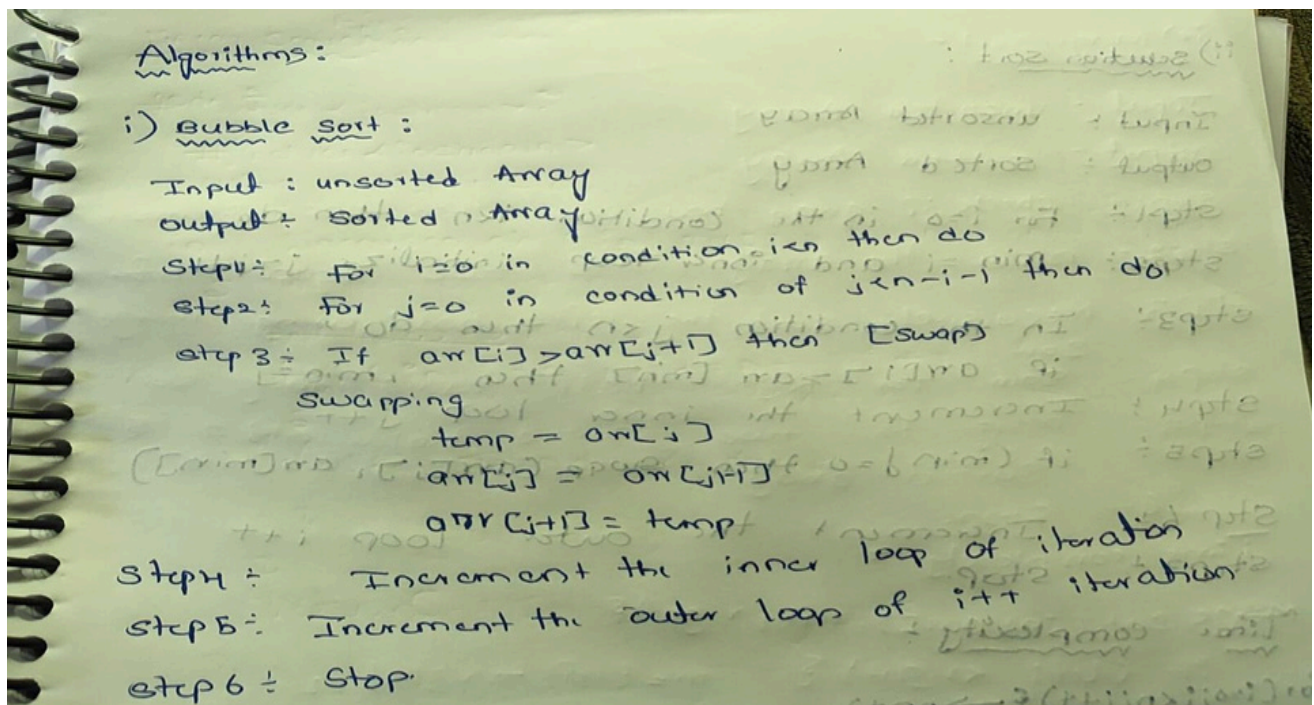
29/07/2026

Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort

Aim: To Implement and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort.

1. Bubblesort:

Algorithm:



Program:

```
#include <iostream>
#include <vector>

using namespace std;

void bubbleSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; ++i) {
        for (int j = 0; j < n - i - 1; ++j) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
```

```
        arr[j] = arr[j + 1];
        arr[j + 1] = temp;
    }
}
}
}

int main() {
    int size;
    cout << "Enter the number of elements: ";
    cin >> size;

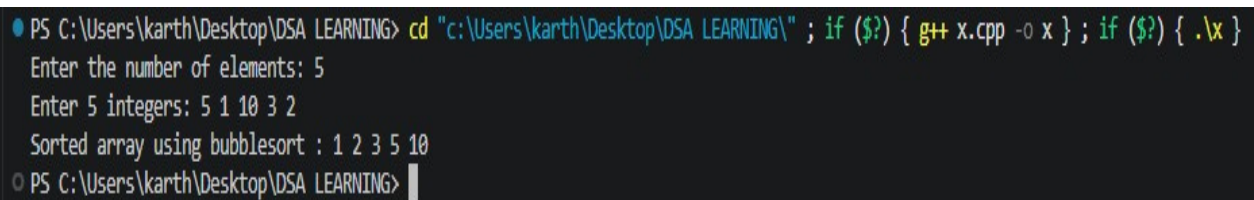
    vector<int> arr(size);
    cout << "Enter " << size << " integers: ";
    for (int i = 0; i < size; ++i) {
        cin >> arr[i];
    }

    bubbleSort(arr);

    cout << "Sorted array: ";
    for (int i = 0; i < size; ++i) {
        cout << arr[i] << " ";
    }
    cout << endl;

    return 0;
}
```

Output:



```
PS C:\Users\karth\Desktop\DSA LEARNING> cd "c:\Users\karth\Desktop\DSA LEARNING\" ; if ($?) { g++ x.cpp -o x } ; if ($?) { .\x }
Enter the number of elements: 5
Enter 5 integers: 5 1 10 3 2
Sorted array using bubblesort : 1 2 3 5 10
PS C:\Users\karth\Desktop\DSA LEARNING>
```

Time Complexity:

Time complexity:

$$\begin{aligned} \text{for } i=0; i < n; i++ &\rightarrow 2n+1 \\ \text{for } j=0; j < n-1-i; j++ &\rightarrow \frac{n(n-1)}{2} \\ \text{if } (a[i] > a[j+1]) &\rightarrow \frac{n(n-1)}{2} \\ t = a[i] &\rightarrow \frac{n(n-1)}{2} \times 3 \\ a[j+1] = a[i] &\rightarrow \frac{n(n-1)}{2} \times 3 \\ a[i] = t &\rightarrow \frac{n(n-1)}{2} \times 3 \end{aligned}$$

$$\text{Total } T(n) = (2n+1) + \frac{n(n-1)}{2} + \frac{n(n-1)}{2} + \frac{n(n-1)}{2} + \frac{n(n-1)}{2} + \frac{n(n-1)}{2} + \frac{n(n-1)}{2}$$

$$= 2n+1 + \frac{5n(n-1)}{2}$$

$$= \frac{4n+2 + 5n^2 - 5n}{2}$$

$$= \frac{5n^2 - n + 2}{2}$$

$$T(n) = O(n^2)$$

2. Selection sort:

Algorithm:

ii) Selection sort:

Input: unsorted Array
Output: sorted Array

Step 1: For $i=0$ in the condition $i < n$ then do two

Step 2: $\text{min} = i$ and inner loop initialize $j = i+1$

Step 3: In the condition $j < n$ then do,
if $a[j] < a[\text{min}]$ then $\text{min} = j$

Step 4: Increment the inner loop $j++$

Step 5: if $\text{min} \neq i$ then swap $a[i]$ and $a[\text{min}]$

Step 6: Increment the outer loop $i++$

Step 7: stop

Program:

```
#include <iostream>
#include <vector>

void selectionSort(std::vector<int>&arr) {
    int n = arr.size();
    for(int i=0; i<n-1; ++i){
        int minIndex = i;
        for(int j=i+1; j<n; ++j){
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        if (minIndex != i) {
            std::swap(arr[i], arr[minIndex]);
        }
    }
}

int main() {
    int n;
    std::cout << "Enter the number of elements: ";
    std::cin >> n;

    if(n <= 0) {
        return 0;
    }

    std::vector<int> arr(n);
    std::cout << "Enter " << n << " integers: ";
    for (int i = 0; i < n; ++i) {
        std::cin >> arr[i];
    }

    selectionSort(arr);

    std::cout << "Sorted array: ";
    for (int i = 0; i < n; ++i) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;

    return 0;
}
```


Output:

```
PS C:\Users\karth\Desktop\DSA LEARNING> cd "c:\Users\karth\Desktop\DSA LEARNING\" ; if ($?) { g++ x.cpp -o x } ; if ($?) { .\x }
Enter the number of elements: 4
Enter 4 integers: 0 1 5 2
Sorted array using selection sort: 0 1 2 5
PS C:\Users\karth\Desktop\DSA LEARNING>
```

Time complexity:

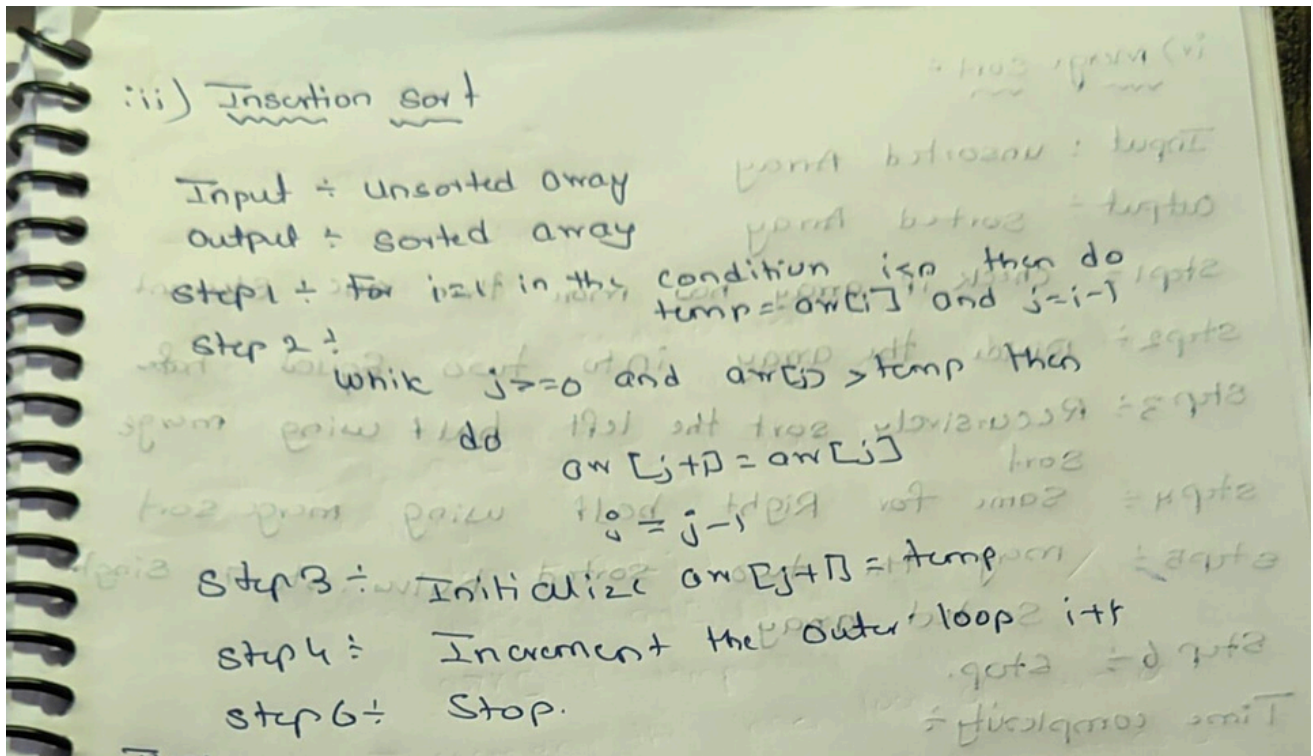
Time complexity:

For $(i=0; i < n; i++) \rightarrow 2n+2$
 $\min = i \rightarrow n$
 For $(j=i+1; j < n; j++) \rightarrow n(n-1)$
 If $(arr[i] < arr[\min]) \rightarrow n(n-1)$
 $\min = j \rightarrow n(n-1)$
 3
 If $(\min \neq 0) \rightarrow n(n-1)$
 $t = arr[i]$
 $arr[i] = arr[\min]$
 $arr[\min] = t$
 3

$T(n) = 2n+2 + n \times \frac{n(n-1)}{2}$
 $T(n) = 2n+2 + \frac{n^3 - n^2}{2}$
 $T(n) = \frac{2n^3 - 2n^2 + 4n + 2}{2}$
 $T(n) = \frac{2n^3 - 2n^2 + 4n + 2}{2}$
 $T(n) = O(n^3)$

Insertion sort:

Algorithm:



Program:

```
#include <iostream>
#include <vector>
void insertionSort(std::vector<int>& arr) {
    int n = arr.size();
    for(int i=1; i<n; ++i) {
        int key = arr[i];
        int j=i-1;
        while(j>=0 && arr[j] > key) {
            arr[j+1]=arr[j];
            j=j-1;
        }
        arr[j + 1] = key;
    }
}

int main() {
    int n;
    std::cout<<"Enter the number of elements: ";
    std::cin >> n;
```

```

if(n <= 0) {
    return 0;
}

std::vector<int> arr(n);
std::cout << "Enter " << n << " integers: ";
for(int i = 0; i < n; ++i) {
    std::cin >> arr[i];
}

insertionSort(arr);

std::cout << "Sorted array: ";
for(int i = 0; i < n; ++i) {
    std::cout << arr[i] << " ";
}
std::cout << std::endl;

return 0;
}

```

Output:

```

PS C:\Users\karth\Desktop\DSA LEARNING> cd "c:\Users\karth\Desktop\DSA LEARNING\" ; if ($?) { g++ x.cpp -o x } ; if ($?) { .\x }
Enter the number of elements: 4
Enter 4 integers: 10 2 5 6
Sorted array: 2 5 6 10
PS C:\Users\karth\Desktop\DSA LEARNING>

```

Time complexity:

Time Complexity

For $(i=0; i < n; i++)$ $\rightarrow 1+n+(n-1) \dots 1$ (sum of first n natural numbers) $\rightarrow \frac{n(n+1)}{2}$

$i = arr[i]$ $\rightarrow 1$

$j = i-1$ $\rightarrow 1$

while $(j >= 0 \text{ \& \& } arr[j] > arr[i])$ $\rightarrow \frac{n(n-1)}{2}$

$arr[j+1] = arr[i];$ $\rightarrow n(n-1)$

$j = j-1$ $\rightarrow 1$

$arr[i+1] = arr[i];$ $\rightarrow 1$

$T(n) = 2n + 2(n-1) + \frac{n(n-1)}{2} + n(n-1) + (n-1)$

$= 2n + 2n - 2 + \frac{n(n-1)}{2} + n(n-1) + n - 1$

$= 3n - 1 + 2n + \frac{n(n-1)}{2} + n(n-1)$

$= 3n - 1 + 2n + \frac{n^2 - n}{2} + n^2 - n$

$= n^2 - n + n$

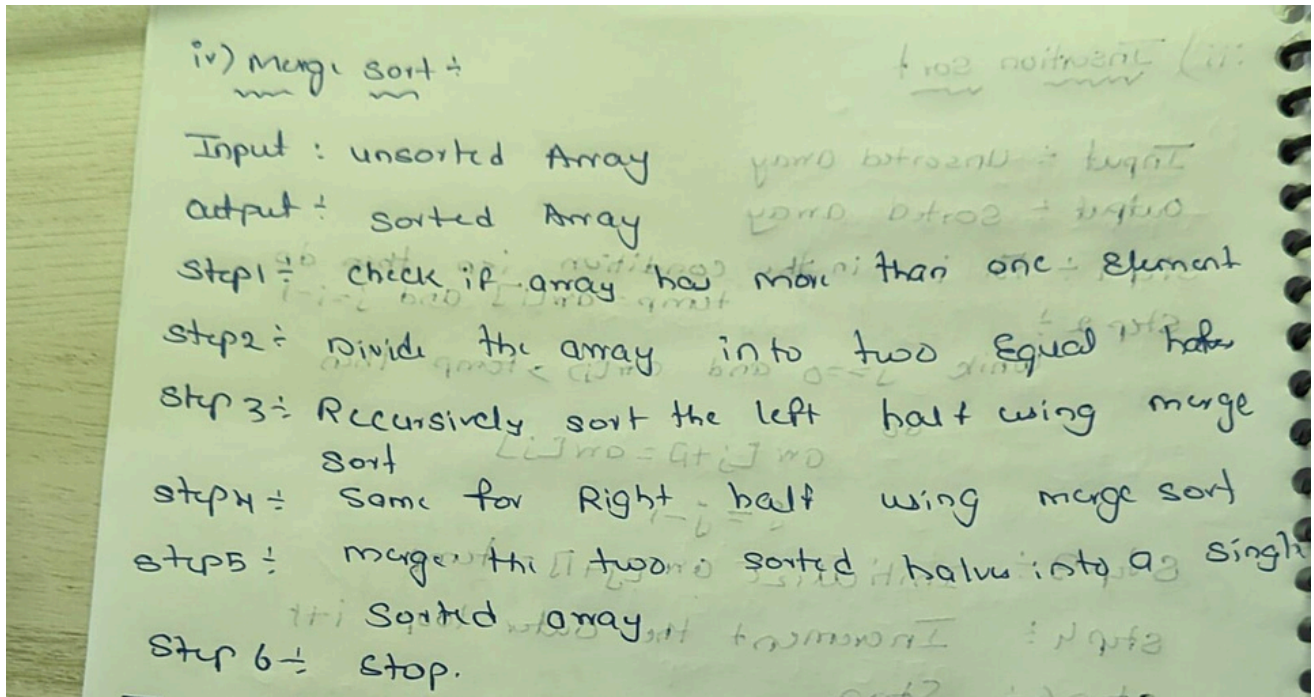
$= n^2$

$T(n) = O(n^2)$

Time complexity $T(n) = O(n^2)$

4. Merge sort:

Algorithm:



Program:

```
#include <iostream>
```

```
#include <vector>
```

```
void merge(std::vector<int>& arr, int left, int mid, int right) {
```

```
    int n1 = mid - left + 1;
```

```
    int n2 = right - left;
```

```
    std::vector<int> L(n1);
```

```
    std::vector<int> R(n2);
```

```
    for (int i = 0; i < n1; ++i) {
```

```
        L[i] = arr[left + i];
```

```
    }
```

```
    for (int j = 0; j < n2; ++j) {
```

```
        R[j] = arr[mid + 1 + j];
```

```
    }
```

```
    int i = 0;
```



```
int j = 0;
int k = left;

while(i < n1 && j < n2) {
    if (L[i] <= R[j]) {
        arr[k] = L[i];
        ++i;
    } else {
        arr[k] = R[j];
        ++j;
    }
    ++k;
}

while (i < n1) {
    arr[k] = L[i];
    ++i;
    ++k;
}

while (j < n2) {
    arr[k] = R[j];
    ++j;
    ++k;
}
}

void mergeSort(std::vector<int>& arr, int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

int main() {
    int n;
    std::cout << "Enter the number of elements: ";
    std::cin >> n;

    if (n <= 0) {
        return 0;
    }
}
```

```
std::vector<int> arr(n);
std::cout << "Enter " << n << " integers: ";
for(int i = 0; i < n; ++i) {
    std::cin >> arr[i];
}

mergeSort(arr, 0, n - 1);

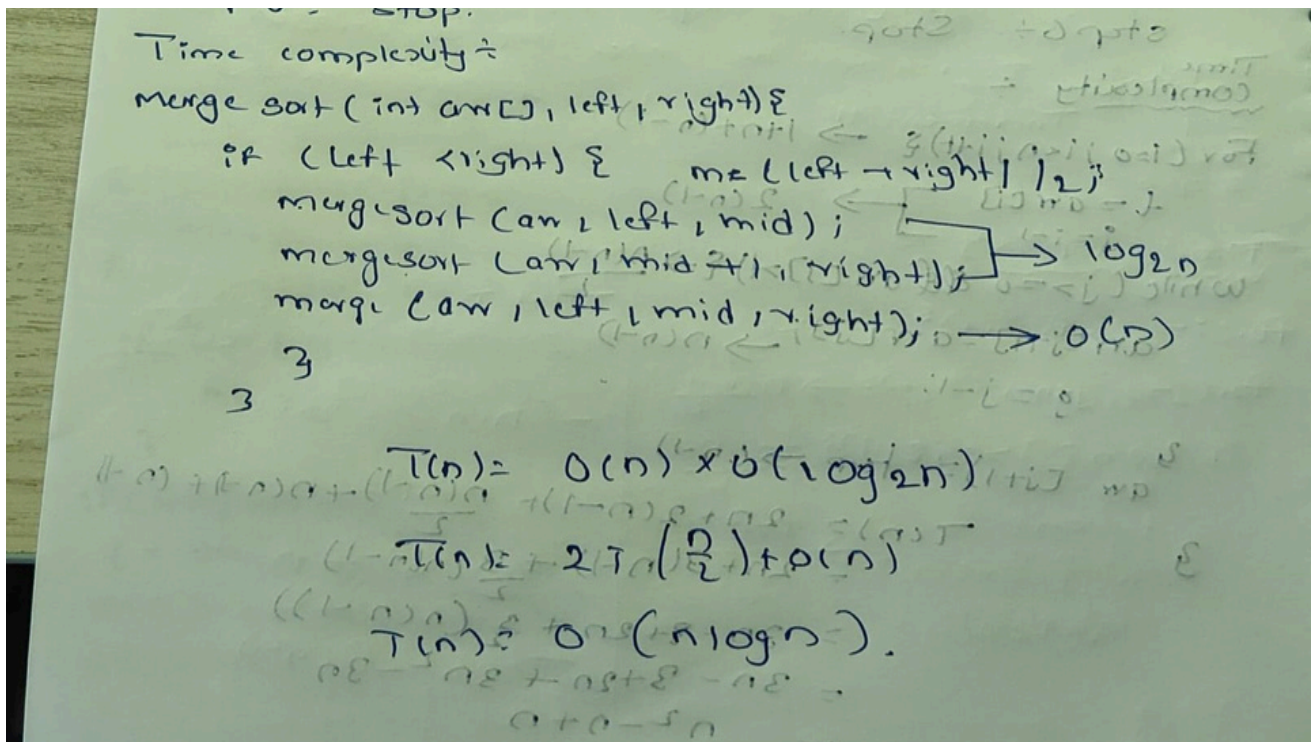
std::cout << "Sorted array: ";
for(int i = 0; i < n; ++i) {
    std::cout << arr[i] << " ";
}
std::cout << std::endl;

return 0;
}
```

Output:

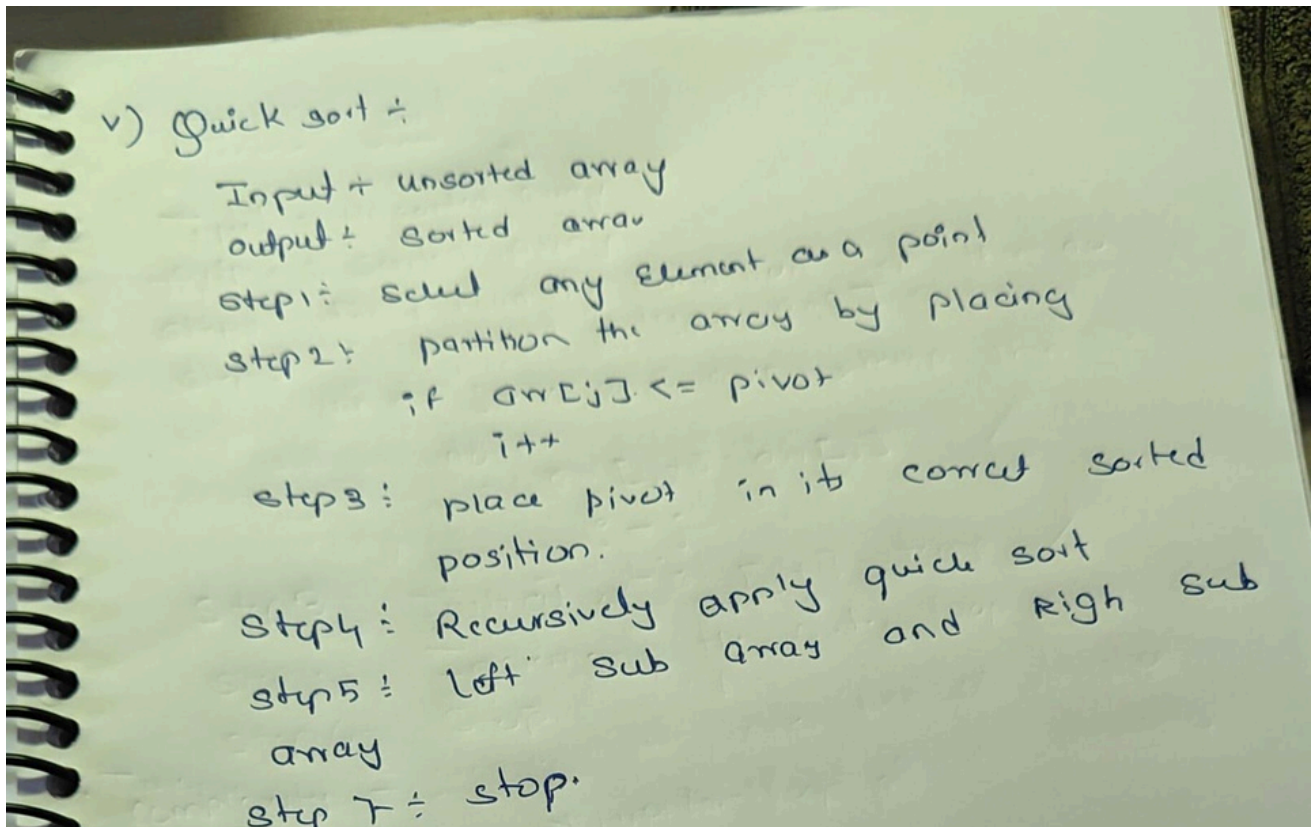
```
PS C:\Users\karth\Desktop\DSA LEARNING> cd "c:\Users\karth\Desktop\DSA LEARNING\" ; if ($?) { g++ x.cpp -o x } ; if ($?) { .\x }
Enter the number of elements: 4
Enter 4 integers: 10 2 5 1
Sorted array: 1 2 5 10
PS C:\Users\karth\Desktop\DSA LEARNING>
```

Time complexity:



5. Quick sort:

Algorithm:



Program:

```
#include <iostream>
#include <vector>
void swap(int& a, int& b) {
    int temp = a;
    a = b;
    b = temp;
}

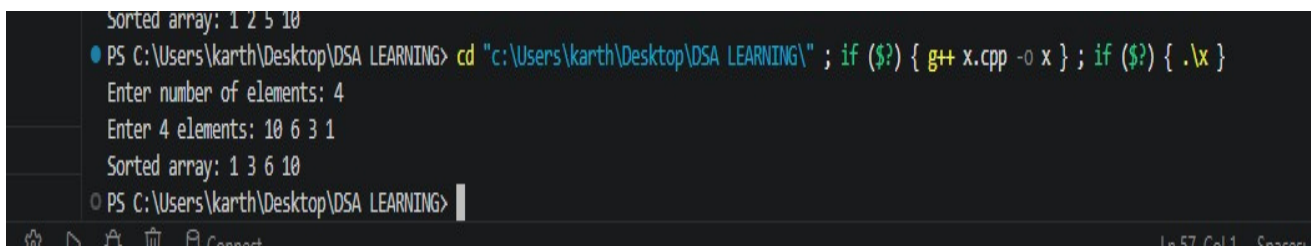
int partition(std::vector<int>& arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
```

```
for(int j=low; j< high; j++) {
    if(arr[j]<pivot) {
        i++;
        swap(arr[i],arr[j]);
    }
}
swap(arr[i+1],arr[high]);
return i + 1;
}

void quickSort(std::vector<int>& arr, int low, int high) {
    if (low < high) {
        int pi=partition(arr, low, high);
        quickSort(arr,low, pi - 1);
        quickSort(arr,pi + 1, high);
    }
}

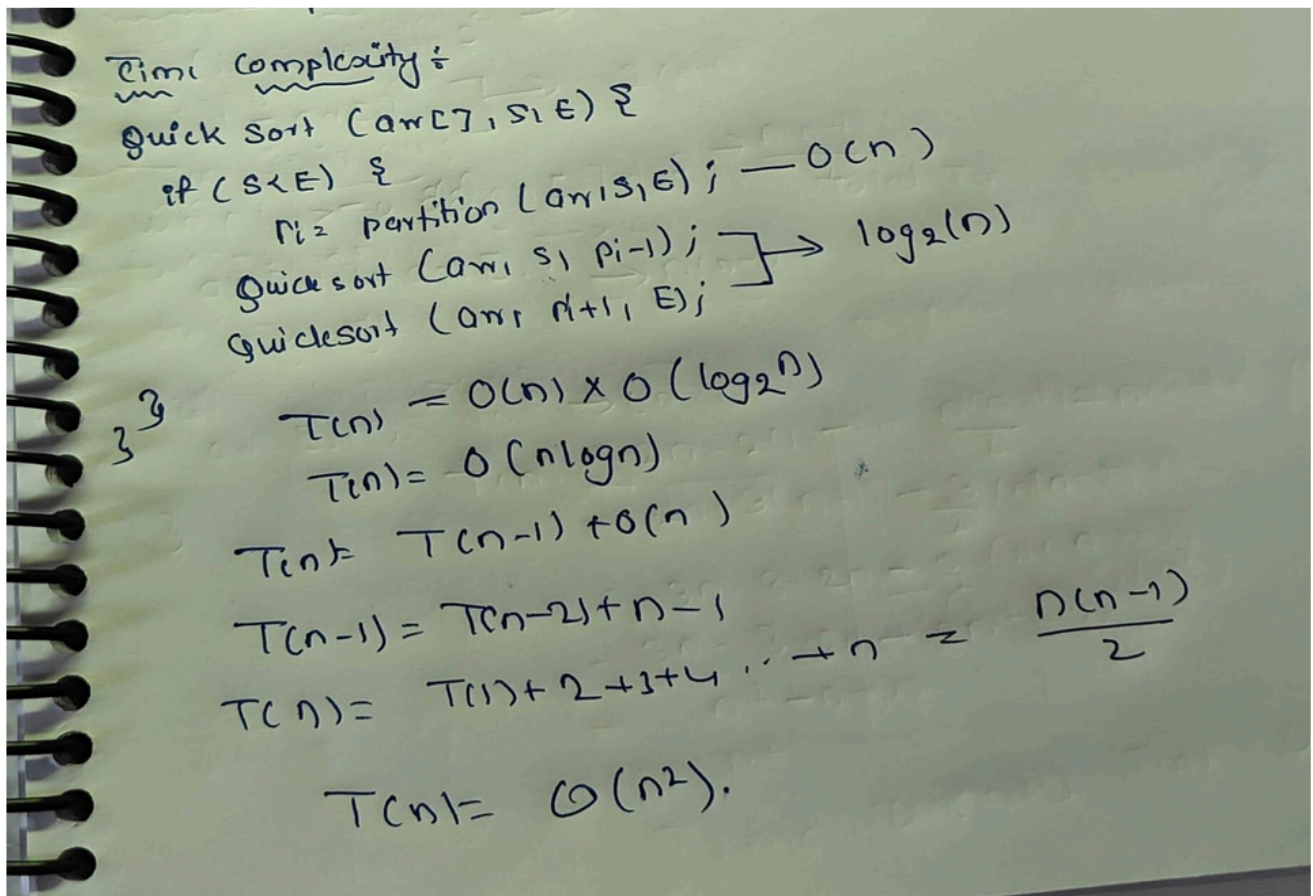
int main() {
    int n;
    std::cout<<"Enter number of elements: ";
    std::cin >> n;

    std::vector<int> arr(n);
    std::cout<<"Enter " << n << " elements: ";
    for(int i=0; i<n; i++) {
        std::cin>>arr[i];
    }
    quickSort(arr,0,n - 1);
    std::cout<<"Sorted array: ";
    for(int i=0; i<n; i++) {
        std::cout<<arr[i] << " ";
    }
    std::cout<<std::endl;
    return 0;
}
```

Output:

```
Sorted array: 1 2 5 10
PS C:\Users\karth\Desktop\DSA LEARNING> cd "c:\Users\karth\Desktop\DSA LEARNING\" ; if ($?) { g++ x.cpp -o x } ; if ($?) { .\x }
Enter number of elements: 4
Enter 4 elements: 10 6 3 1
Sorted array: 1 3 6 10
PS C:\Users\karth\Desktop\DSA LEARNING>
```


Time complexity:



conclusion :

Hence, By Implementing and analysing of sorting algorithms.
 Bubble sort, Selection sort, Insertion sort, Merge sort and
 Quicksort.

The time complexity plays an important role in the logical part
 of the program. And the Programs are successfully executed.